

Process Description and Control

How the OS represents each process, and how it steers its life cycle

01 DESCRIPTION

PROCESS CONTROL BLOCK

the OS's complete record of one process

running**IDENTIFICATION**

Process ID 1042
Parent PID 1001
User / Group 37 / 100

PROCESSOR STATE

Program Counter 0x00A3F0
CPU Registers R0 ... R15
Stack Pointer 0x7FF8C0

PROCESS CONTROL

State Running
Priority 4
Memory Limits 0x4000-0x6000
Open Files 3

ALSO TRACKED

I/O status · accounting · scheduling params · IPC

02 CONTROL



OS Management of Application Execution

- Resources are made available to multiple applications.
- The processor is switched among multiple applications so all will appear to be progressing.
- The processor and I/O devices can be used efficiently.

Program vs. Process

- *A recipe is a program. Cooking the dish is a process.*
- A **program** is passive: it's just a file on disk, a set of instructions sitting there doing nothing.
- A **process** is a program *brought to life*.

What is a process?

- An instance of a program running on a computer.
 - User's point of view
- The entity that can be assigned to and executed on a processor.
 - OS scheduler's point of view
- A unit of activity characterized by the execution of a sequence of instructions, a current state, and an associated set of system resources.
 - OS implementer's point of view

Process elements

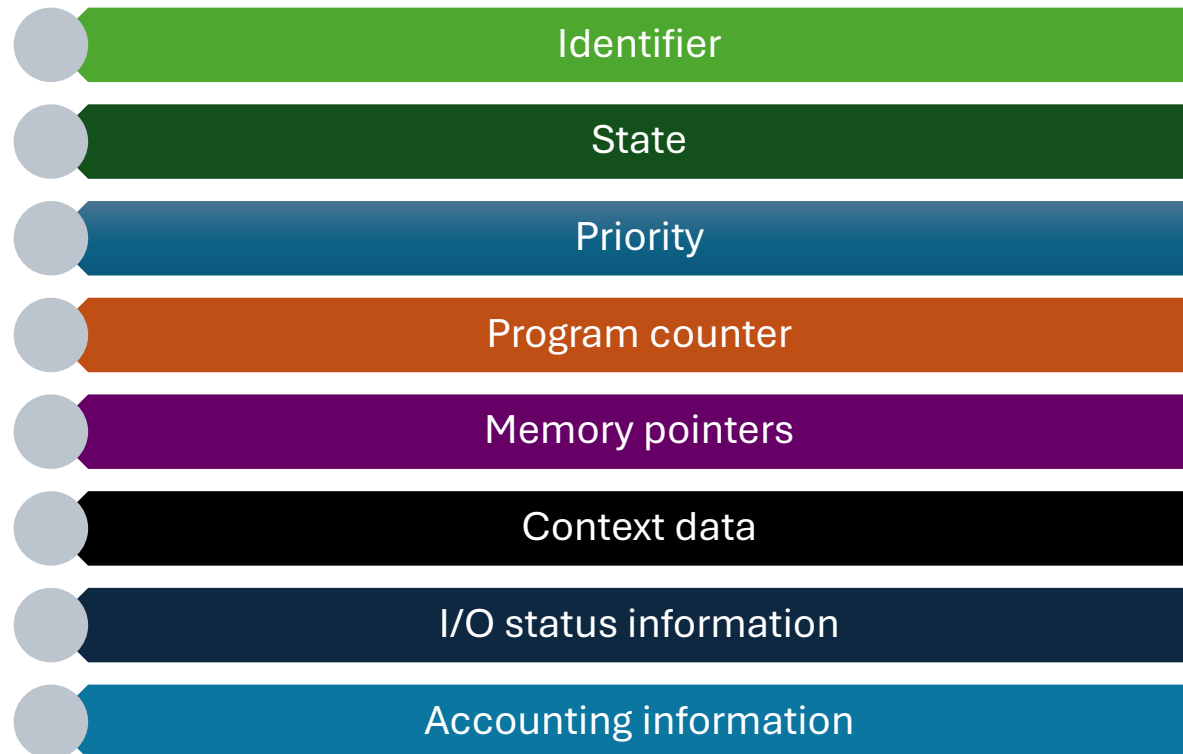
Program code

- which may be shared with other processes that are executing the same program

A set of data associated with that code

Process Elements

- While the program is executing, this process can be uniquely characterized by a number of elements, including:



Why it matter?

- It matters because the **OS has to manage many processes at once**, and without these elements, it wouldn't be able to:
 - Distinguish processes
 - Suspend and resume correctly
 - Provide isolation and protection
 - Coordinate resources – I/O and scheduling info lets the OS fairly share devices, files, and CPU time among processes.
- The process elements **allow the OS to treat each process as a distinct, manageable unit**, enabling multitasking, protection, and efficient resource use.

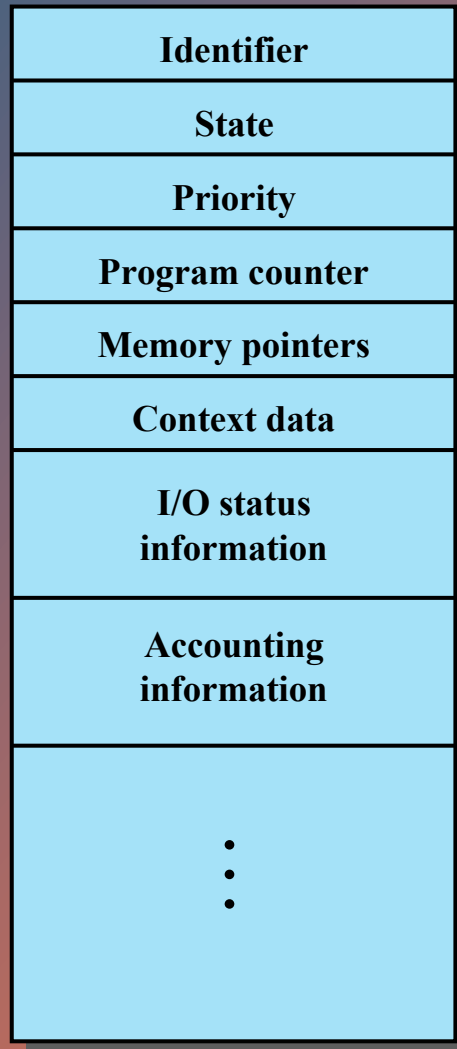


Figure 3.1 Simplified Process Control Block

Process control block (PCB)

- The OS stores these elements for each process in a dedicated data structure: the **Process Control Block (PCB)**.
- OS creates one PCB for each process and manages it for the life of that process.
- PCB contains enough information to interrupt a running process and later resume the execution of the process as if interruption has never occurred.

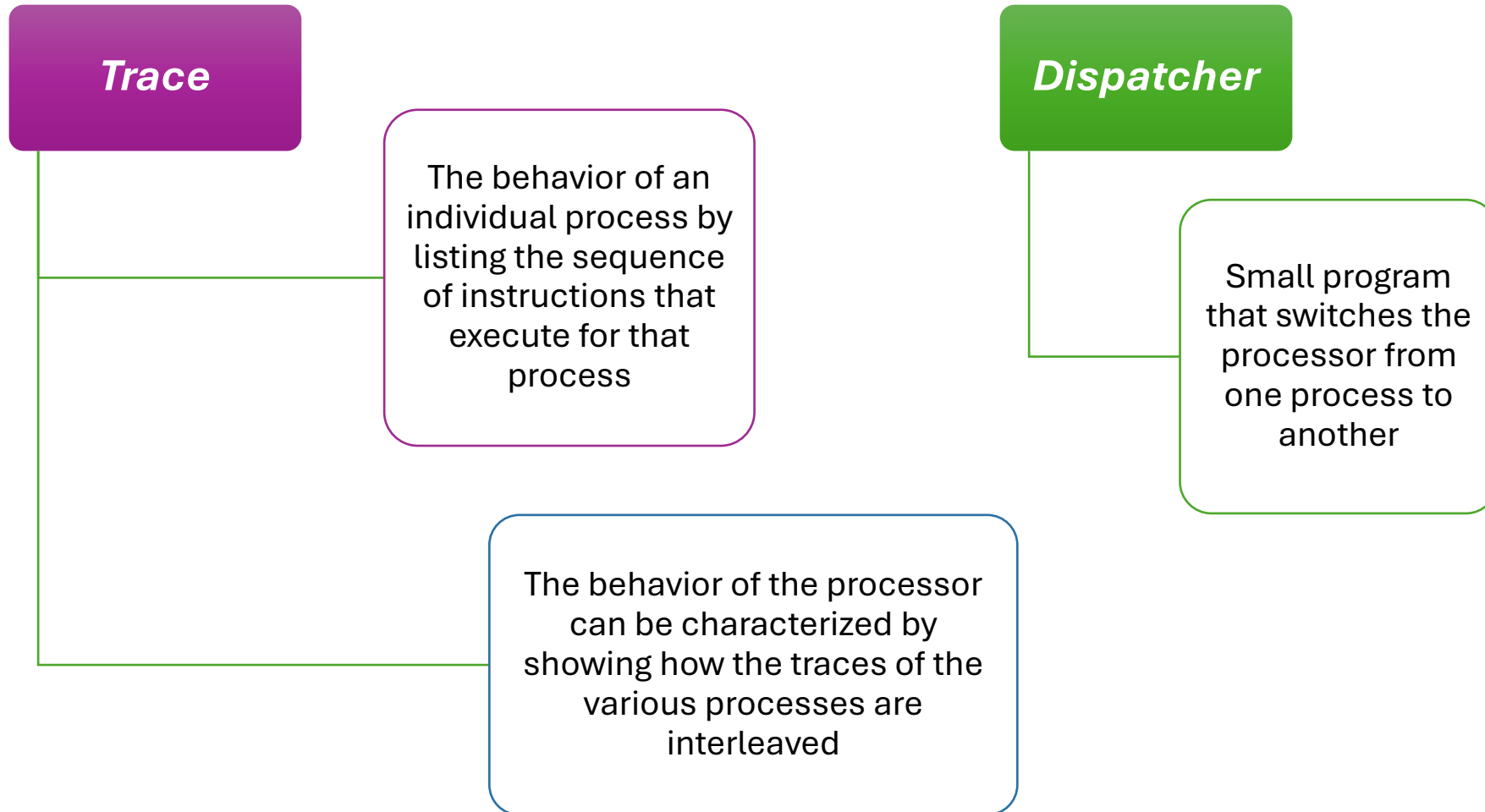
What happens when a process is interrupted?

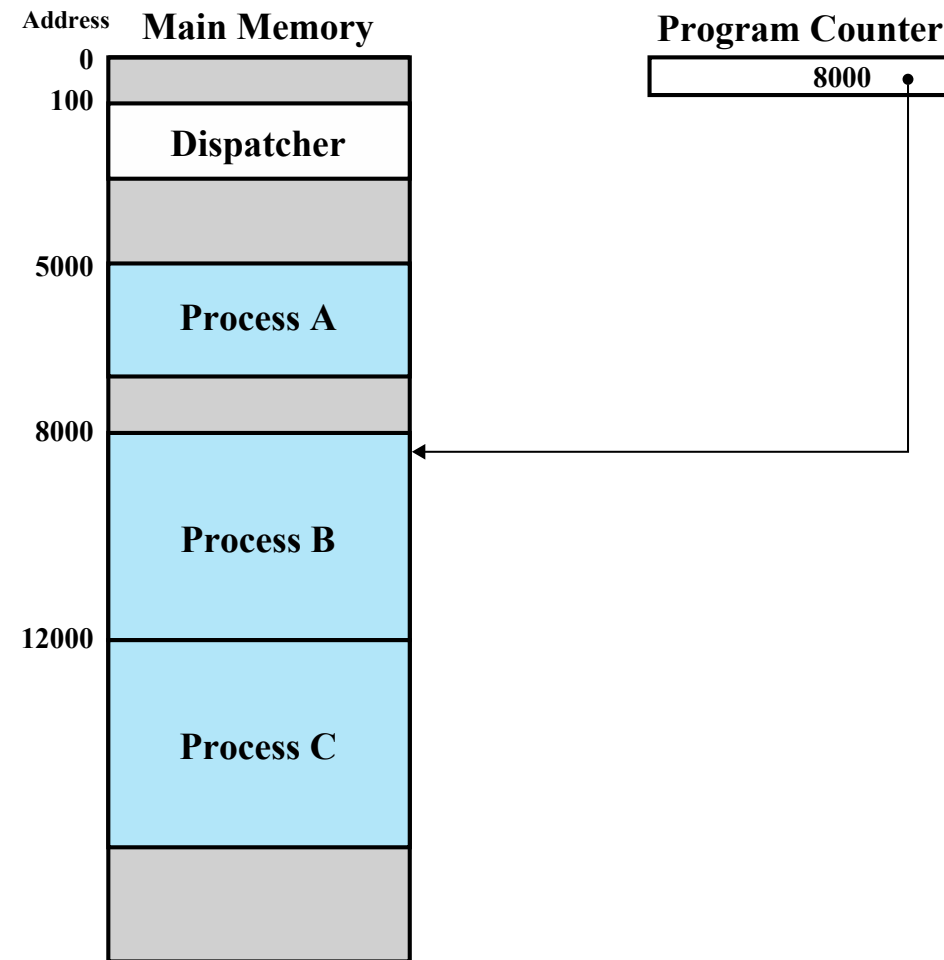
- The current values of the program counter and the processor registers (context data) are saved in the appropriate fields in PCB.
- process control block, and the state of the process is changed to some other value, such as blocked or ready.
- Then the OS is free to put some other process in the running state.

Process states

- We can characterize the behavior of an individual **process** by listing the sequence of instructions that execute for that process. Such a listing is referred to as a trace of the process.
- We can also characterize behavior of the processor (not to be confused with process) by showing how the traces of the various processes are interleaved.

Trace and Dispatcher: Tracking and Switching Processes





**Figure 3.2 Snapshot of Example Execution (Figure 3.4)
at Instruction Cycle 13**

**From each of the
processes' point of
view**

5000	8000	12000
5001	8001	12001
5002	8002	12002
5003	8003	12003
5004		12004
5005		12005
5006		12006
5007		12007
5008		12008
5009		12009
5010		12010
5011		12011

(a) Trace of Process A

(b) Trace of Process B

(c) Trace of Process C

5000 = Starting address of program of Process A

8000 = Starting address of program of Process B

12000 = Starting address of program of Process C

Figure 3.3 Traces of Processes of Figure 3.2

From the processors
point of view

1	5000			27	12004
2	5001			28	12005
3	5002			----- Timeout	
4	5003			29	100
5	5004			30	101
6	5005			31	102
----- Timeout				32	103
7	100			33	104
8	101			34	105
9	102			35	5006
10	103			36	5007
11	104			37	5008
12	105			38	5009
13	8000			39	5010
14	8001			40	5011
15	8002			----- Timeout	
16	8003			41	100
----- I/O Request				42	101
17	100			43	102
18	101			44	103
19	102			45	104
20	103			46	105
21	104			47	12006
22	105			48	12007
23	12000			49	12008
24	12001			50	12009
25	12002			51	12010
26	12003			52	12011
				----- Timeout	

100 = Starting address of dispatcher program

Shaded areas indicate execution of dispatcher process;
first and third columns count instruction cycles;
second and fourth columns show address of instruction being executed

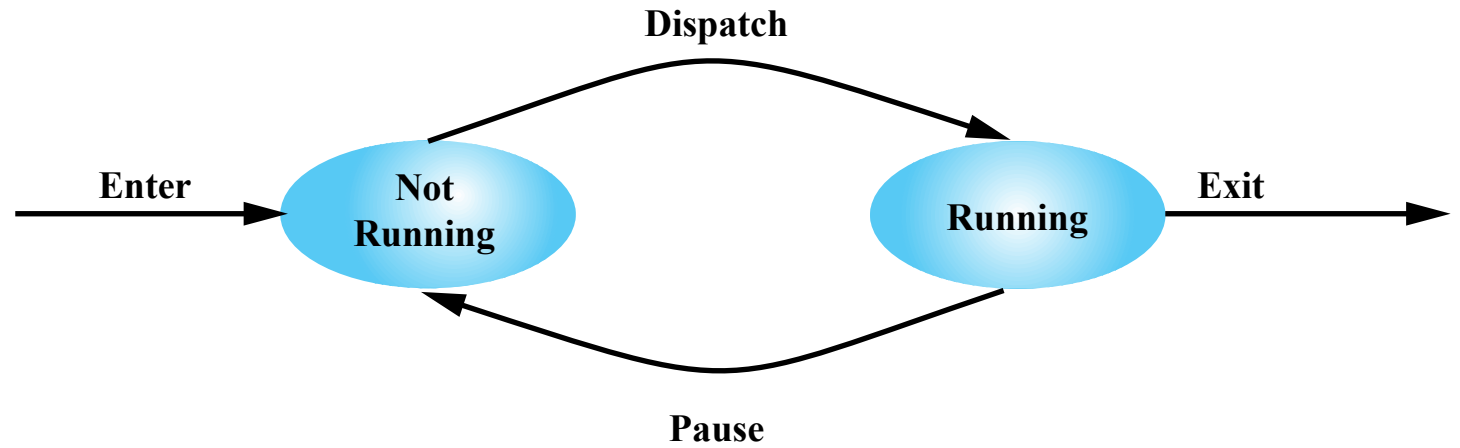
Figure 3.4 Combined Trace of Processes of Figure 3.2

<https://www.cs.montana.edu/adiesha/animations/assets/os/process-trace.html>

Two-State Process model

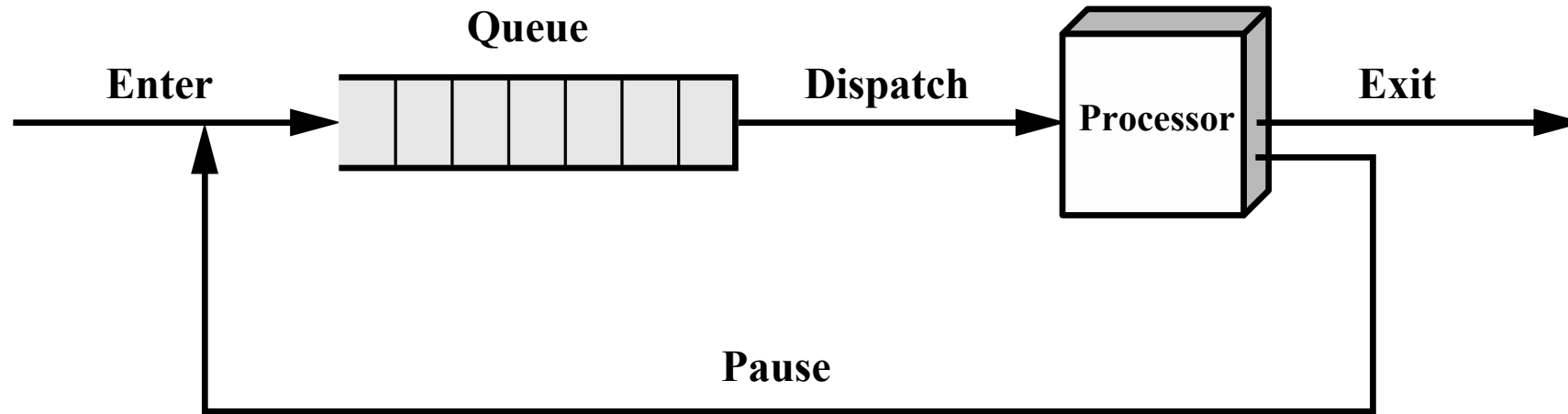
- The OS's main job is controlling process execution — deciding the interleaving pattern and allocating resources.
- To manage that, we need a simple way to describe what a process is doing at any moment.
- The simplest model: a process is either ***Running*** (currently on the processor) or ***Not Running*** (everything else).

Two-state process model



(a) State transition diagram

- PCB can be used to keep track of the process information—where the process is in the memory, current state of the process.
- When a process is not being executed it will stay in the queue, waiting for its turn.



(b) Queuing diagram

Figure 3.5 Two-State Process Model

- Before refining this simple model, let's look at the creation and termination of processes.

Creating and Termination of processes

- When a new process is added to those currently being managed, the OS builds the necessary data structure to handle them—namely the PCB.
- In a batch environment, a process is created in response to the submission of a job.
- In an interactive environment, a process is created when a new user attempts to log on.
- OS is responsible for creation of a new process.
- OS may also create processes on behalf of an application.
 - Process spawning
 - Parent process.
 - Child process.

Creating and Termination of processes

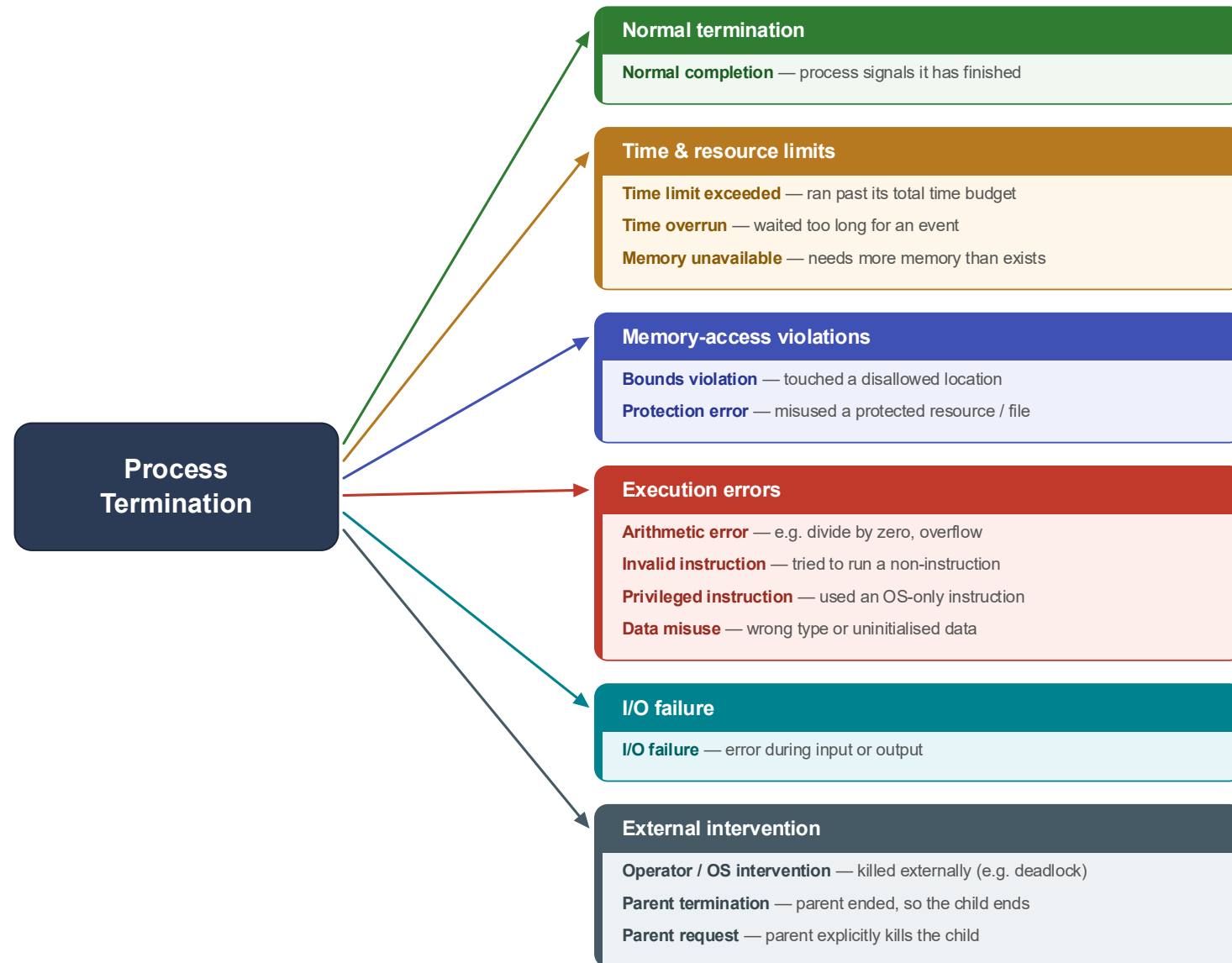
New batch job	The OS is provided with a batch job control stream, usually on tape or disk. When the OS is prepared to take on new work, it will read the next sequence of job control commands.
Interactive log-on	A user at a terminal logs on to the system
Created by OS to provide a service	The OS can create a process to perform a function on behalf of a user program, without the user having to wait (e.g., a process to control printing).
Spawned by existing process	For purposes of modularity or to exploit parallelism, a user program can dictate the creation of a number of processes

Process Termination

- There must be a way to terminate a process and inform it to the user.
- In batch systems, typically job includes a halt instruction or an explicit OS service call for termination.
 - This generates an interrupt to the processor which invokes the necessary part of the OS to deal with the job completion.
- In a time-sharing system, when user logs off or shuts off the terminal a process may terminate.
- In a workstation, when the user quits an application, a process may terminate.
- A process may terminate on errors or fault conditions.

Reasons for Process Termination

How and why a running process comes to an end



Limitations of the simple two-state model

- Not all processes are ready to execute — a process that has issued an I/O request must wait for it to finish before it can run again.
- So, the "Not Running" state actually hides two different situations: processes that are *ready* to execute, and processes that are *blocked* waiting for an event.
- With a single queue for all not-running processes, the dispatcher can't just take the next one — it may have to search the queue for a process that is actually ready to run.
- The two-state model can't express this difference, so it can't manage it. We need a better model.

Five-State model

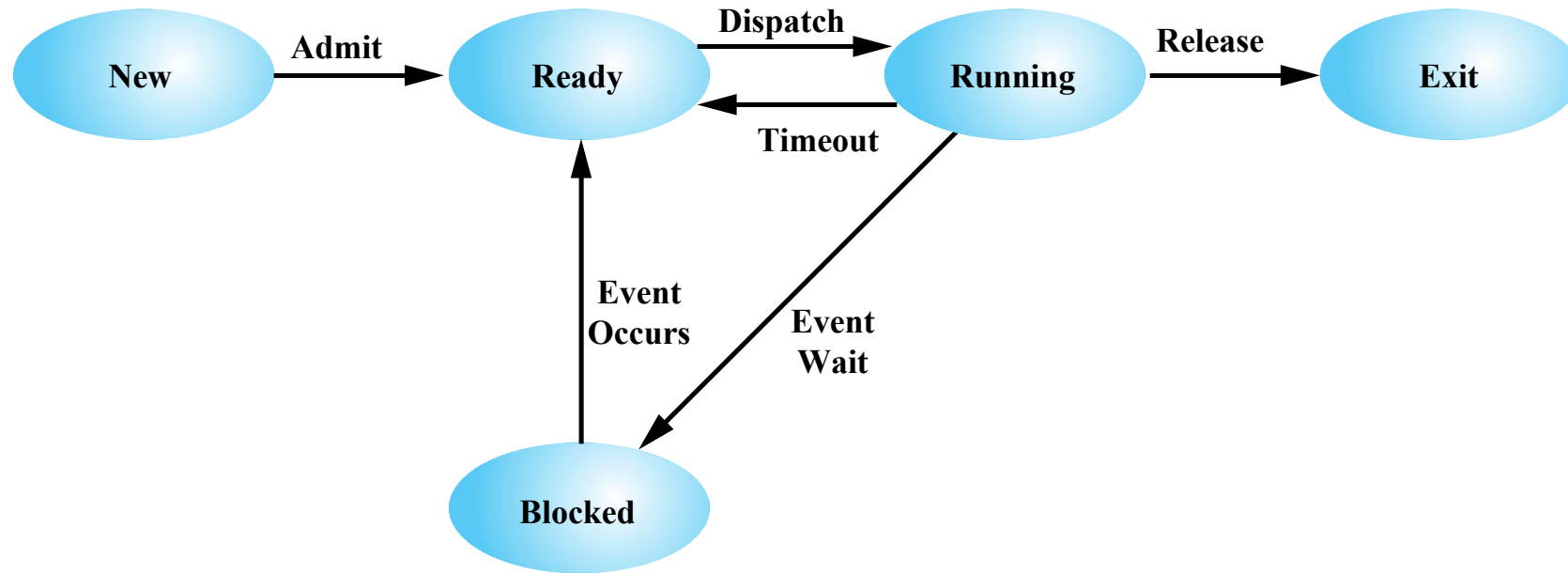


Figure 3.6 Five-State Process Model

Five-State model

- Let's look at the transitions.

- **Null -> New:** A new process is created to execute a program.

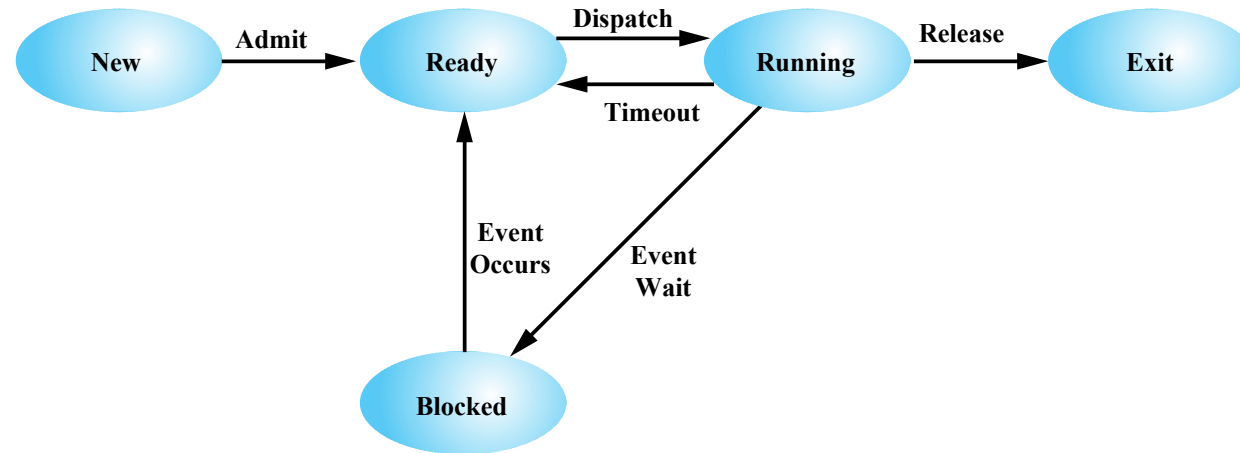


Figure 3.6 Five-State Process Model

- **New -> Ready:** The OS will move a process from the New state to the **Ready state** when it is prepared to take on an additional process.
- Most systems set some limit based on the number of existing processes or the amount of virtual memory committed to existing processes.

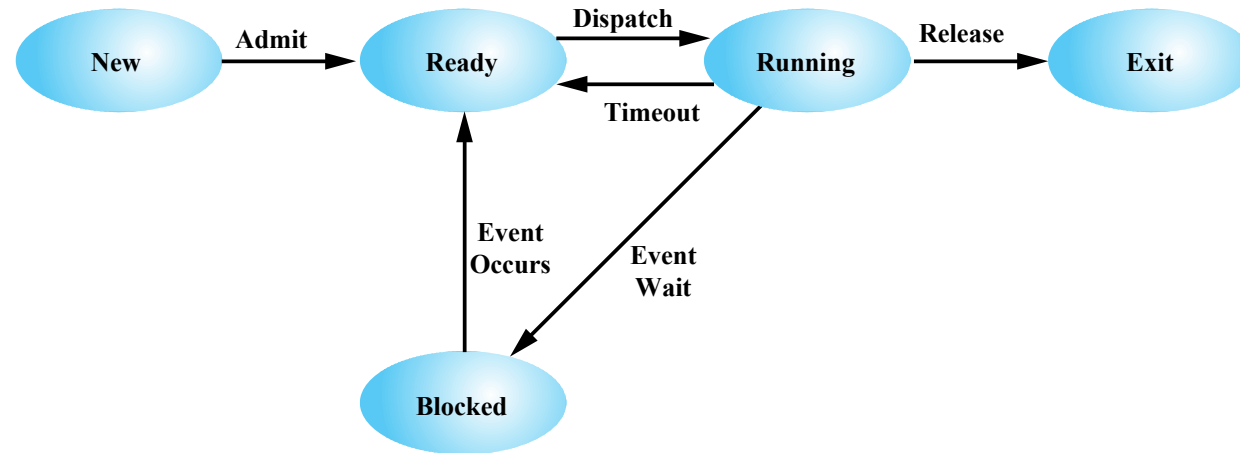


Figure 3.6 Five-State Process Model

- **Ready -> Running: Dispatcher selects** a process to run when the system is ready for a process to run.

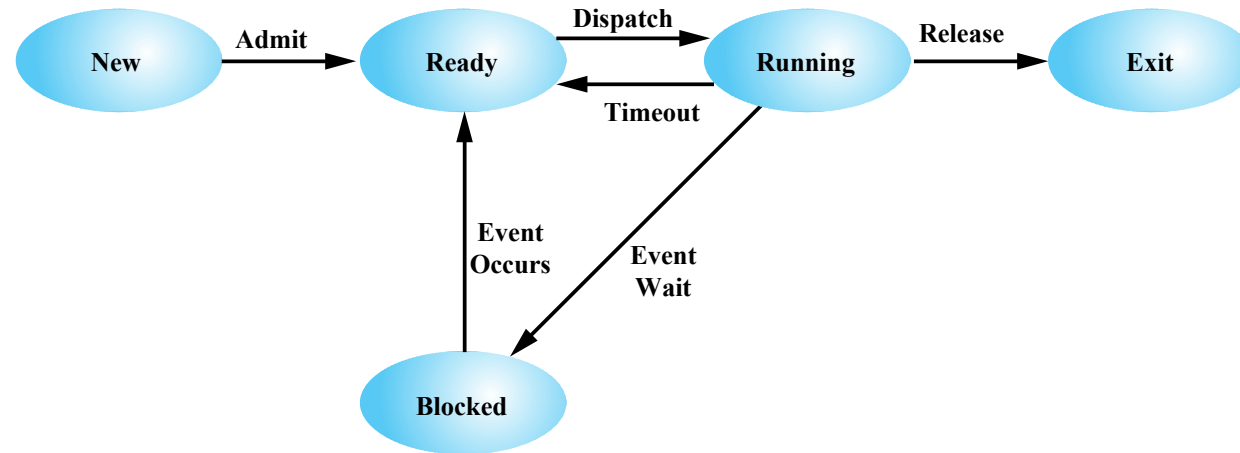


Figure 3.6 Five-State Process Model

- **Running -> Exit:** The currently running process is terminated by the OS if the process indicates that it has completed or if it aborts

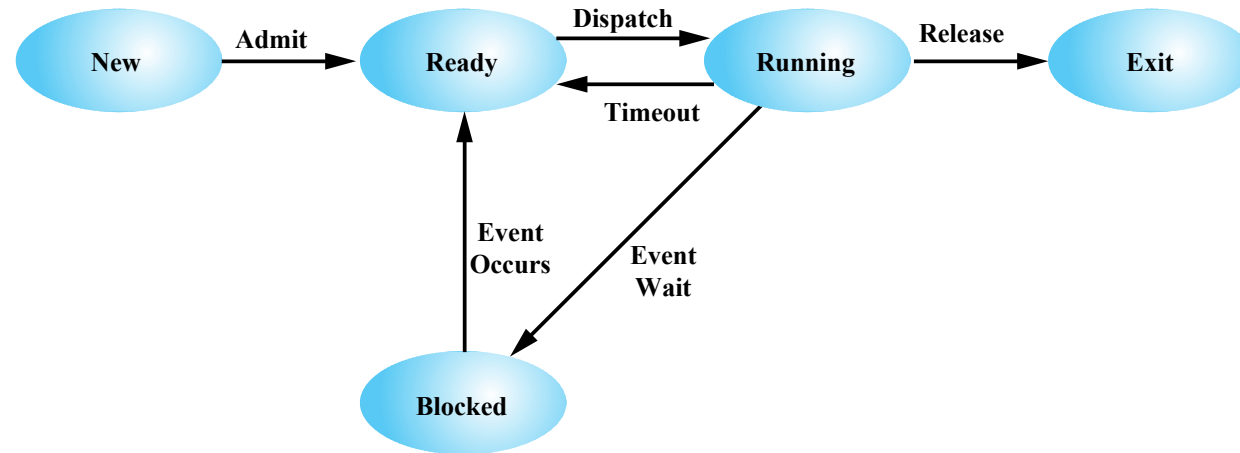


Figure 3.6 Five-State Process Model

- **Running -> Ready:**

- When maximum allowed time is reached.
- Priority based situation.

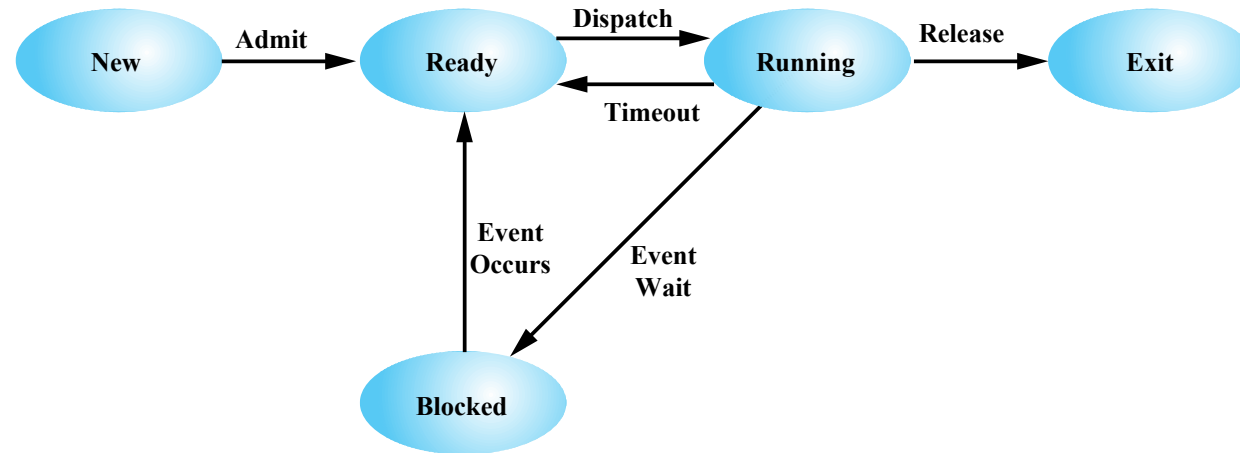


Figure 3.6 Five-State Process Model

- **Running -> Blocked:** A process is put in the **Blocked state** if it requests something for which it must wait.
 - System call
 - I/O operation
 - Inter process communication

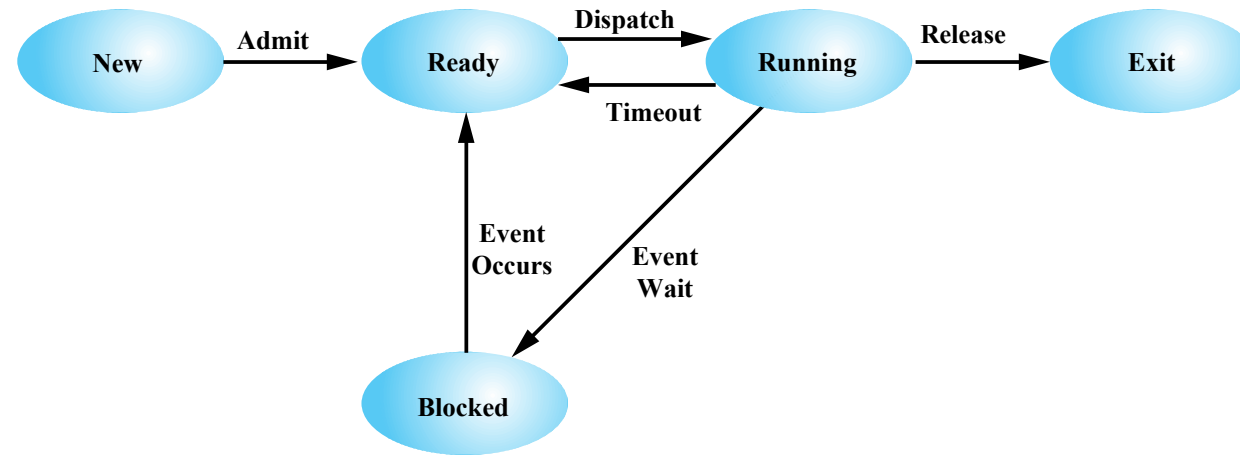


Figure 3.6 Five-State Process Model

- **Blocked -> Ready:** A process in the Blocked state is moved to the Ready state when the event for which it has been waiting occurs.

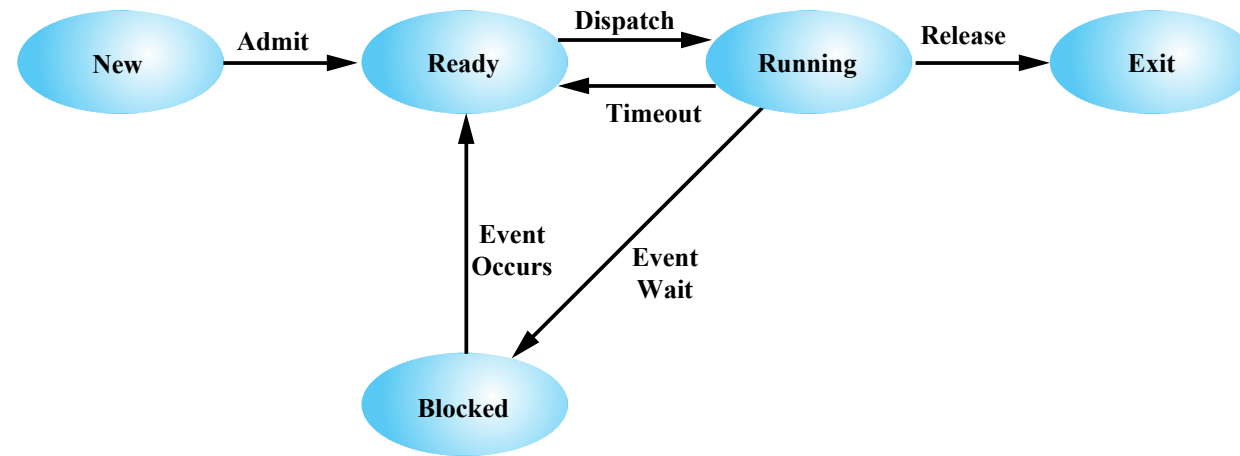


Figure 3.6 Five-State Process Model

- **Ready -> Exit:** In some systems, a parent may terminate a child process at any time. Also, if a parent terminates, all child processes associated with that parent may be terminated.
- **Blocked -> Exit:** same reasons as above

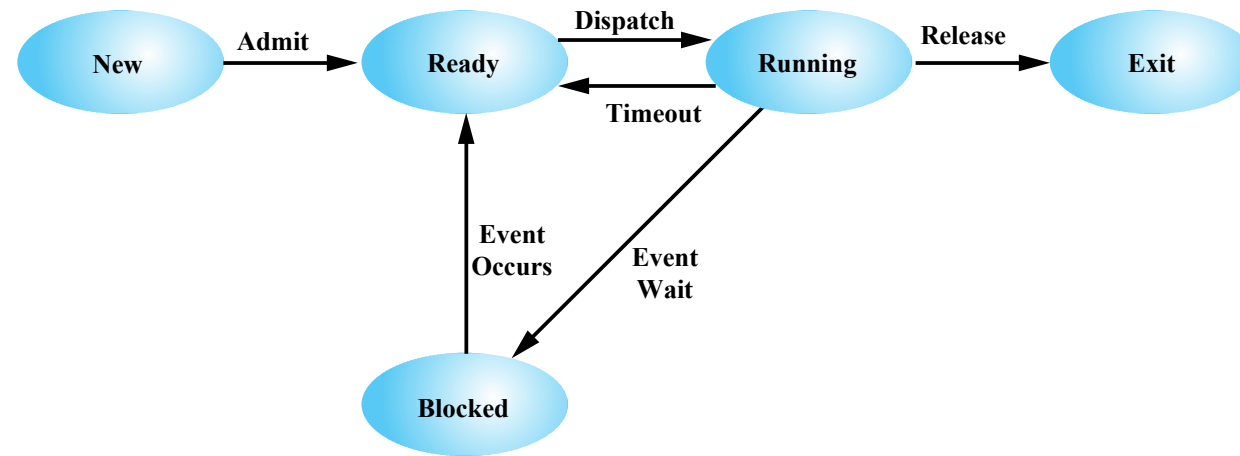


Figure 3.6 Five-State Process Model

<https://www.cs.montana.edu/adiesha/animations/assets/os/process-trace.html>

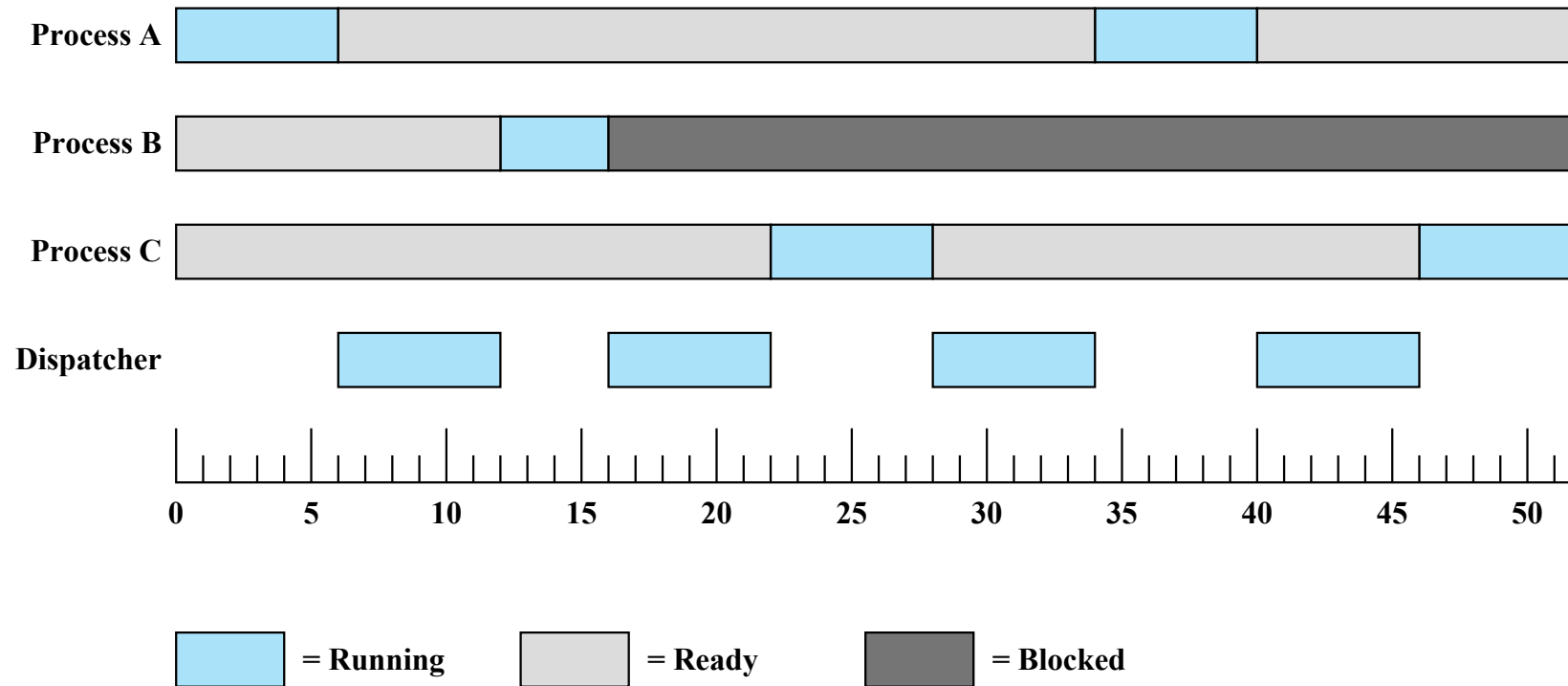
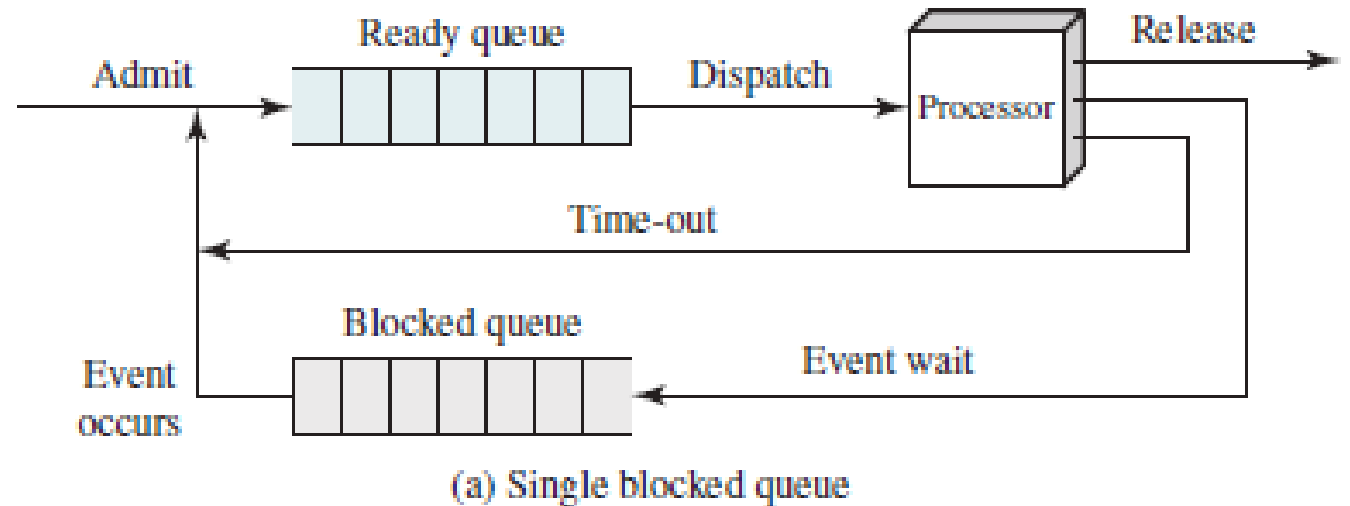


Figure 3.7 Process States for Trace of Figure 3.4

Implementation ideas

- How can we use the notion of blocked state and develop a better system?

- Simple approach.
- Any disadvantages?
- when an event occurs, the OS must scan the entire blocked queue, searching for those processes waiting on that event.
 - There could be thousands of processes at a given time waiting for an I/O event.
 - E.g., disk read completes.



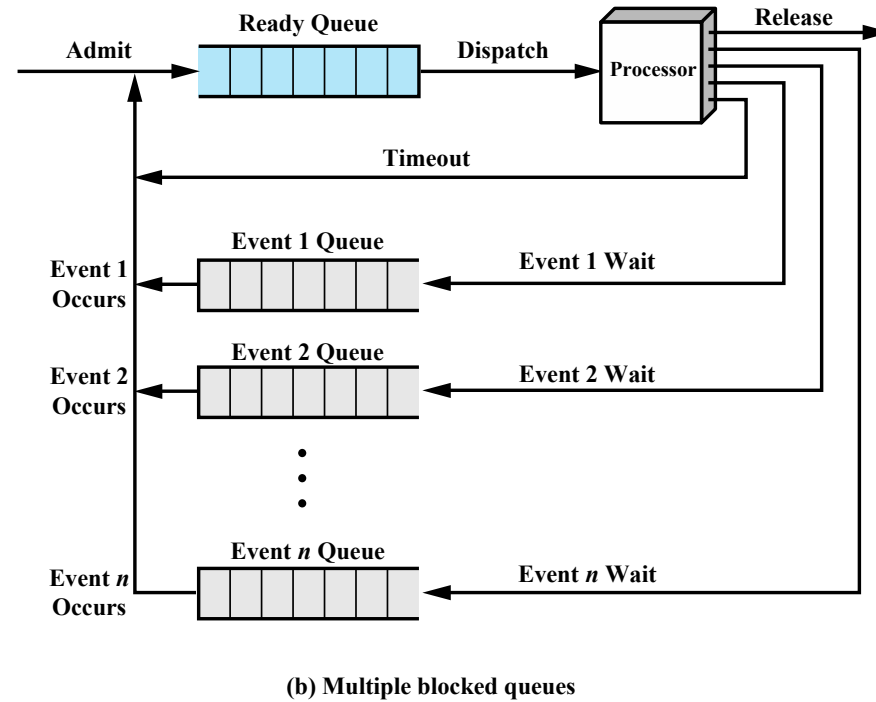
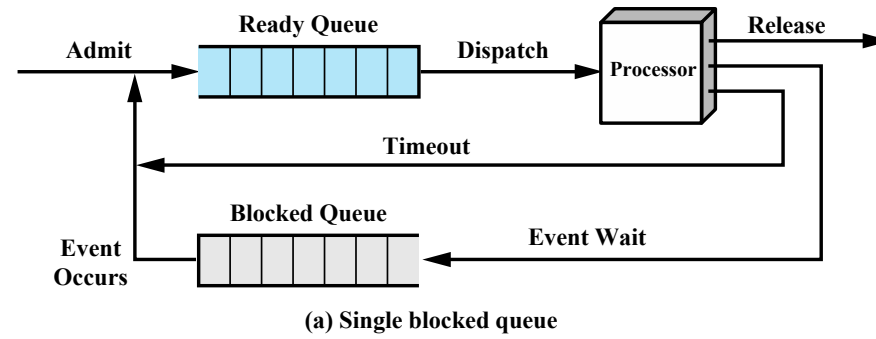


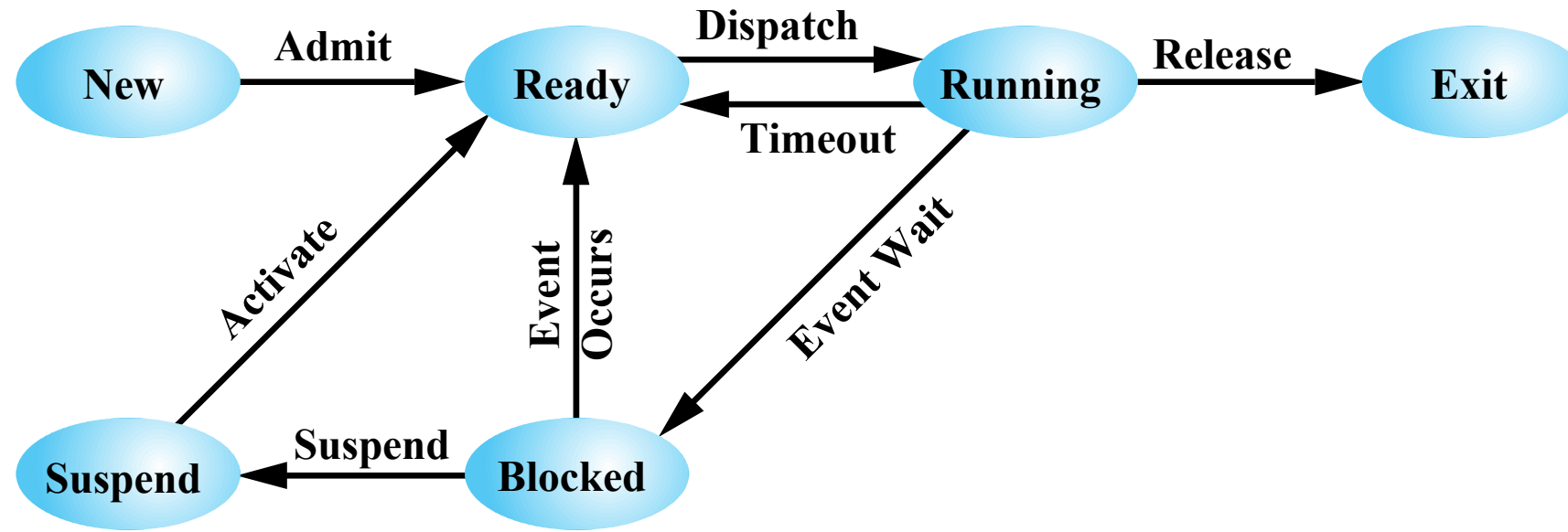
Figure 3.8 Queuing Model for Figure 3.6

What if we use swapping

- What if all processes in main memory are blocked and has no empty space (or low on space?)
- Maybe we need more states to handle concepts of virtual memory.

Suspended processes

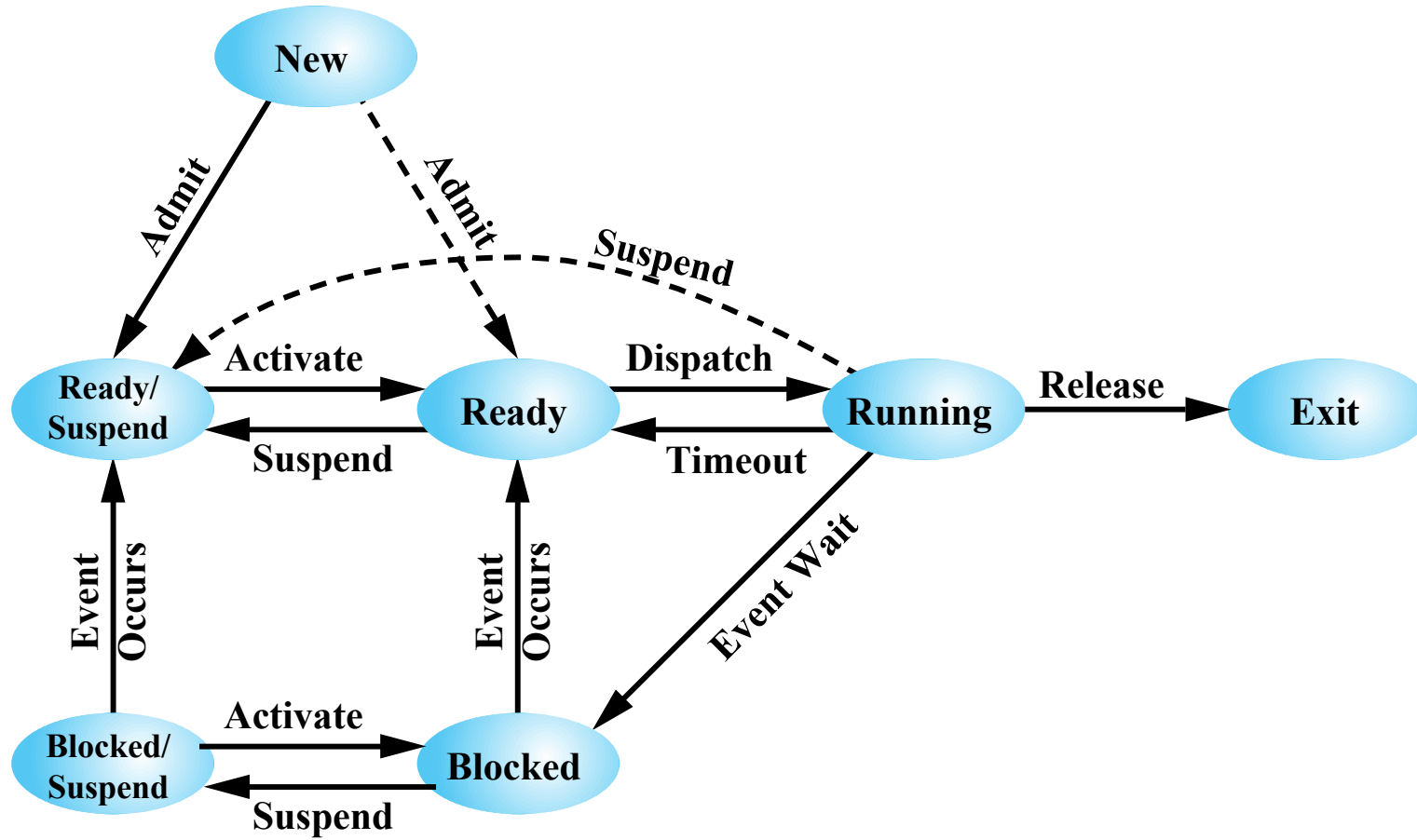
- Swapping
 - Moves (part or all) of a process from main memory to secondary storage.
 - Concept in virtual memory.
 - When none of the processes in main memory is in the Ready state, the OS swaps one of the blocked processes out on to disk into a suspend queue
 - This is a queue of existing processes that have been temporarily kicked out of main memory, or suspended
 - The OS then brings in another process from the suspend queue or it honors a new-process request
 - Execution then continues with the newly arrived process
- We can add Suspend state to support this.



(a) With One Suspend State

We can improve this further

- Suspended process may still be waiting for event to occur.
- Event that the suspended process was waiting may have occurred already.
- Note that if we use virtual memory, we can execute a process which is partially in the main memory.
- But even with VM, OS may have to swap the whole process.



(b) With Two Suspend States

Figure 3.9 Process State Transition Diagram with Suspend States

Characteristics of a suspended process

- The process is not immediately available for execution
- The process may or may not be waiting on an event
- The process was placed in a suspended state by an agent: either itself, a parent process, or the OS, for the purpose of preventing its execution
- The process may not be removed from this state until the agent explicitly orders the removal

Reason for process suspension

Swapping	The OS needs to release sufficient main memory to bring in a process that is ready to execute.
Other OS reason	The OS may suspend a background or utility process or a process that is suspected of causing a problem.
Interactive user request	A user may wish to suspend execution of a program for purposes of debugging or in connection with the use of a resource.
Timing	A process may be executed periodically (e.g., an accounting or system monitoring process) and may be suspended while waiting for the next time interval.
Parent process request	A parent process may wish to suspend execution of a descendent to examine or modify the suspended process, or to coordinate the activity of various descendants.

Process description

- Operating system Control Structure
- What information does the OS need to control processes and manage resources for them?

- What information does the OS need to control processes and manage resource for them?

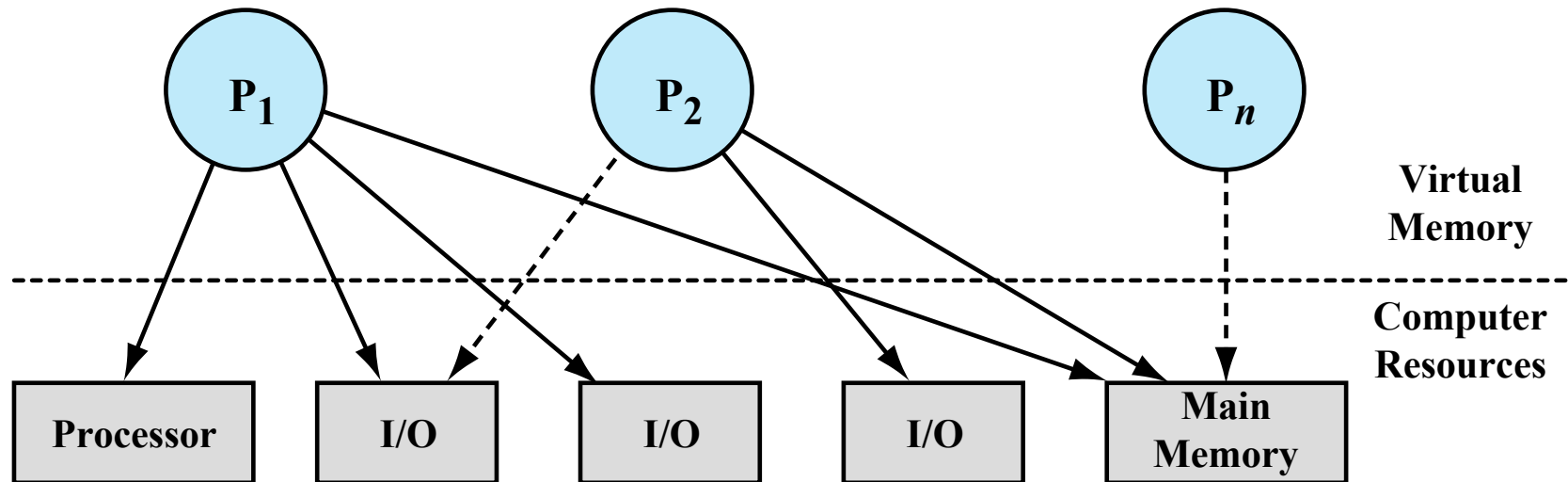


Figure 3.10 Processes and Resources (resource allocation at one snapshot in time)

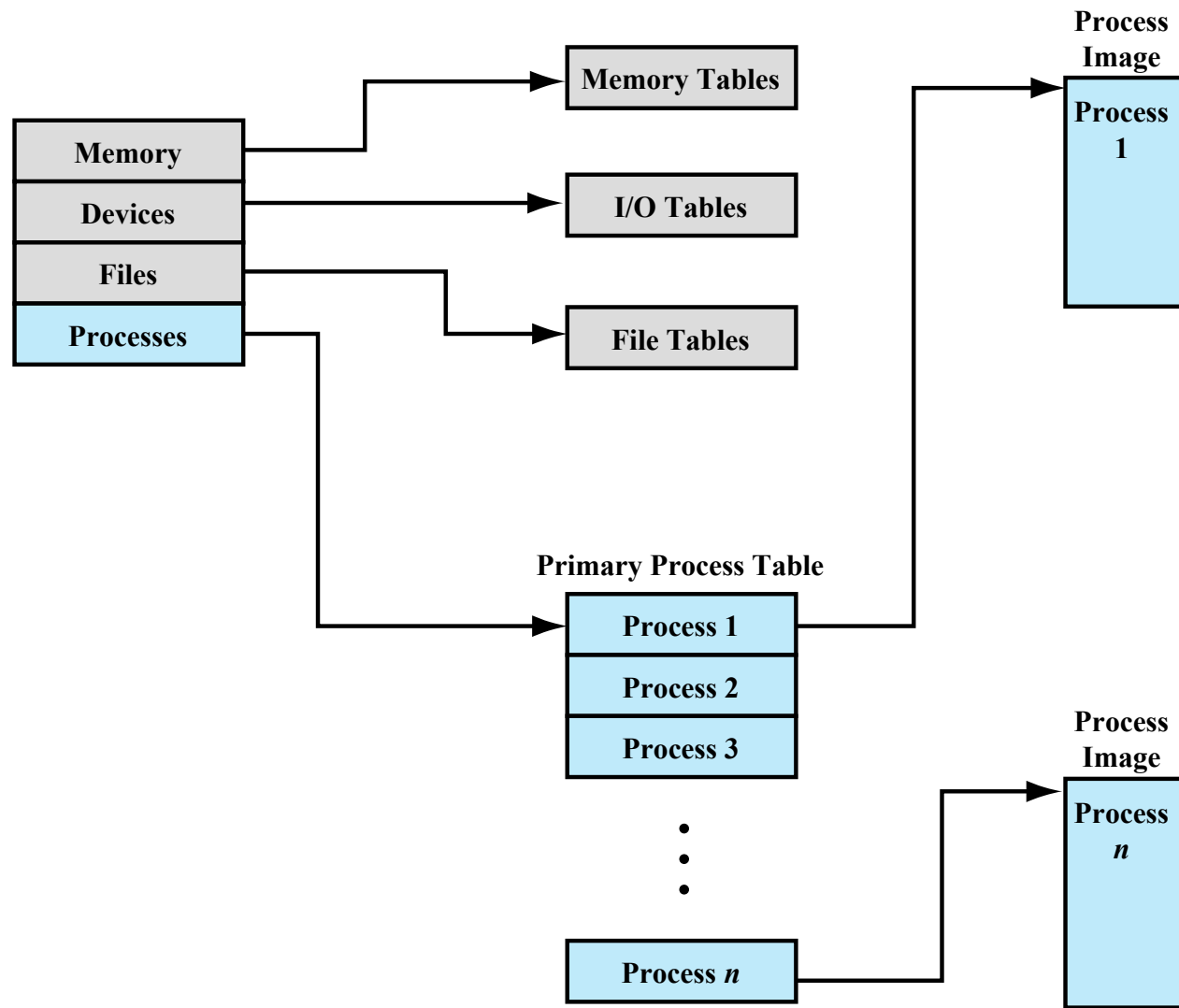


Figure 3.11 General Structure of Operating System Control Tables

Memory Tables

- Used to keep track of both main (real) and secondary (virtual) memory
- Processes are maintained on secondary memory using some sort of virtual memory or simple swapping mechanism
- Page table, segment table, swap table, etc.

Must include:

Allocation of main memory to processes

Allocation of secondary memory to processes

Protection attributes of blocks of main or virtual memory

Information needed to manage virtual memory

I/O Tables

- Used by the OS to manage the I/O devices and channels of the computer system.
- At any given time, an I/O device may be available or assigned to a particular process.
- Device status table, I/O buffer table, etc.
- Keep tracks of processes who requests I/O devices.

If an I/O operation is in progress, the OS needs to know:

- The status of the I/O operation
- The location in main memory being used as the source or destination of the I/O transfer

File Tables

- Information may be maintained and used by a file management system
 - In which case the OS has little or no knowledge of files
- In other operating systems, much of the detail of file management is managed by the OS itself
- File descriptor table.

These tables provide information about:

- Existence of files
- Location on secondary memory
- Current status
- Other attributes

Process Table

- Must be maintained to manage processes
- There must be some reference to memory, I/O, and files, directly or indirectly
- The tables themselves must be accessible by the OS and therefore are subject to memory management

Important notes

- Note that these four distinct sets of tables must be linked or cross-referenced in some fashion.
- OS must have some knowledge of the basic environment.

Process Control Structures

To manage
and control
a process
the OS
must know:

- Where the process is located
- The attributes of the process that are necessary for its management

Process Control Structures

- Process Location

- A process must include a program or set of programs to be executed
- A process will consist of at least sufficient memory to hold the programs and data of that process
- The execution of a program typically involves a stack that is used to keep track of procedure calls and parameter passing between procedures

- Process Attributes

- Each process has associated with it a number of attributes that are used by the OS for process control
- collection of attributes is referred to as a process control block

Process Image

- The collection of program, data, stack, and attributes is referred to as the process image
- Process image location will depend on the memory management scheme being used.

Process Image

- User Data
 - The modifiable part of the user space. May include program data, a user stack area, and programs that may be modified.
- User Program
 - The program to be executed.
- Stack
 - Each process has one or more last-in-first-out (LIFO) stacks associated with it. A stack is used to store parameters and calling addresses for procedure and system calls.
- Process Control Block
 - Data needed by the OS to control the process.

Process Image

- Location of the process image will depend on the memory management scheme.
 - Simple case
 - The process image is maintained as a contiguous, or continuous, block of memory.
 - This block can be maintained in the secondary storage.
 - Entire block must be brought to the main memory to execute the process.
 - Virtual memory
 - No need to bring the whole block into the main memory.
 - Load it to the Virtual Memory (OS will figure everything else).

Process Control Block

- We can group process control block information into 3 categories
 - Process Identification
 - Processor State Information
 - Process Control Information

Process Control Block

- We can group process control block information into 3 categories
 - **Process Identification**
 - Processor State Information
 - Process Control Information

Process Identification

- Numeric identifiers that may be stored with the process control block include
 1. Identifier of this process.
 2. Identifier of the process that created this process.
 3. User identifier.

Process Identification

- Each process is assigned a unique numeric identifier.
 - Otherwise, there must be a mapping that allows the OS to locate the appropriate tables based on the process identifier
- Many of the tables controlled by the OS may use process identifiers to cross-reference process tables.
- Memory tables may be organized to provide a map of main memory with an indication of which process is assigned to each region
 - Similar references will appear in I/O and file tables
- When processes communicate with one another, the process identifier informs the OS of the destination of a particular communication
- When processes are allowed to create other processes, identifiers indicate the parent and descendants of each process.

Process Control Block

- We can group process control block information into 3 categories
 - Process Identification
 - **Processor State Information**
 - Process Control Information

Processor state information

- User visible registers.
 - A user-visible register is one that may be referenced by means of the machine language that the processor executes while in user mode.
 - x86 (32-bit): EAX, EBX, ECX, EDX (previously special purpose registers, now general-purpose register)
- Control and status registers
 - Registers that are employed to control the operation of the processor.
 - Ex: PC, condition codes, status information registers.
 - CF, ZF, SF, OF, PF, IF, DF, TF
- Stack pointers
 - Each process has one or more last-in-first-out (LIFO) system stacks associated with it. A stack is used to store parameters and calling addresses for procedure and system calls. The stack pointer points to the top of the stack.

Processor state information

Consists
of the
contents
of
processor
registers

- User-visible registers
- Control and status registers
- Stack pointers

**Program
status
word
(PSW)**

- Contains condition codes plus other status information
- EFLAGS register is an example of a PSW used by any OS running on an x86 processor
- ARM has Current Program Status Register (CPSR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	I D	V I P	V I F	A C	V M	R F	0	N T	I O P L	O F	D F	I F	T F	S F	Z F	0	A F	0	P F	1	C F	

X ID = Identification flag
 X VIP = Virtual interrupt pending
 X VIF = Virtual interrupt flag
 X AC = Alignment check
 X VM = Virtual 8086 mode
 X RF = Resume flag
 X NT = Nested task flag
 X IOPL = I/O privilege level
 S OF = Overflow flag

C DF = Direction flag
 X IF = Interrupt enable flag
 X TF = Trap flag
 S SF = Sign flag
 S ZF = Zero flag
 S AF = Auxiliary carry flag
 S PF = Parity flag
 S CF = Carry flag

S Indicates a Status Flag
 C Indicates a Control Flag
 X Indicates a System Flag
 Shaded bits are reserved

Figure 3.12 x86 EFLAGS Register

Status Flags (condition codes)

AF (Auxiliary carry flag)

Represents carrying or borrowing between half-bytes of an 8-bit arithmetic or logic operation using the AL register.

CF (Carry flag)

Indicates carrying out or borrowing into the leftmost bit position following an arithmetic operation. Also modified by some of the shift and rotate operations.

OF (Overflow flag)

Indicates an arithmetic overflow after an addition or subtraction.

PF (Parity flag)

Parity of the result of an arithmetic or logic operation. 1 indicates even parity; 0 indicates odd parity.

SF (Sign flag)

Indicates the sign of the result of an arithmetic or logic operation.

ZF (Zero flag)

Indicates that the result of an arithmetic or logic operation is 0.

Control Flag

DF (Direction flag)

Determines whether string processing instructions increment or decrement the 16-bit half-registers SI and DI (for 16-bit operations) or the 32-bit registers ESI and EDI (for 32-bit operations).

System Flags (should not be modified by application programs)

AC (Alignment check)

Set if a word or doubleword is addressed on a nonword or nondoubleword boundary.

ID (Identification flag)

If this bit can be set and cleared, this processor supports the CPUID instruction. This instruction provides information about the vendor, family, and model.

RF (Resume flag)

Allows the programmer to disable debug exceptions so that the instruction can be restarted after a debug exception without immediately causing another debug exception.

IOPL (I/O privilege level)

When set, causes the processor to generate an exception on all accesses to I/O devices during protected mode operation.

IF (Interrupt enable flag)

When set, the processor will recognize external interrupts.

TF (Trap flag)

When set, causes an interrupt after the execution of each instruction. This is used for debugging.

NT (Nested task flag)

Indicates that the current task is nested within another task in protected mode operation.

VM (Virtual 8086 mode)

Allows the programmer to enable or disable virtual 8086 mode, which determines whether the processor runs as an 8086 machine.

VIP (Virtual interrupt pending)

Used in virtual 8086 mode to indicate that one or more interrupts are awaiting service.

VIF (Virtual interrupt flag)

Used in virtual 8086 mode instead of IF.

Process Control Block

- We can group process control block information into 3 categories
 - Process Identification
 - Processor State Information
 - **Process Control Information**

Process Control Information

- The additional information needed by the OS to control and coordinate the various active processes.

Process Control Information

- Scheduling and State Information.
 - This is information that is needed by the operating system to perform its scheduling function. Typical items of information include:
 - Process state
 - Priority
 - Scheduling related information: Depends on the scheduling algorithm used by the OS.
 - Event: Event that process is waiting for before it can be resumed.
- Data Structuring
- Interprocess Communication information
- Process privileges
- Memory management
 - Pointers to page/segment tables
- Resource ownership and Utilization.
 - Resources controlled by the process may be indicated, such as opened files.

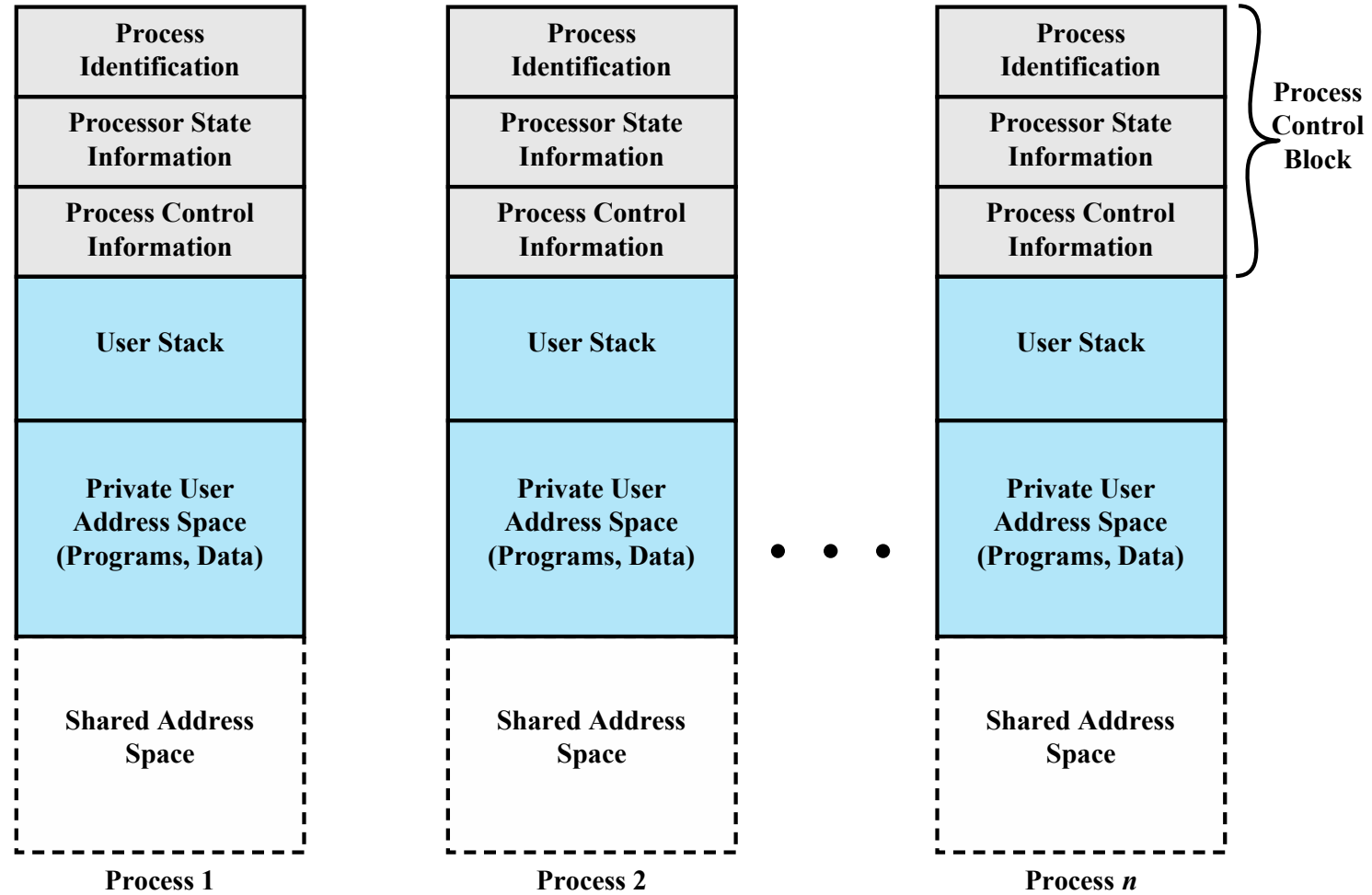


Figure 3.13 User Processes in Virtual Memory

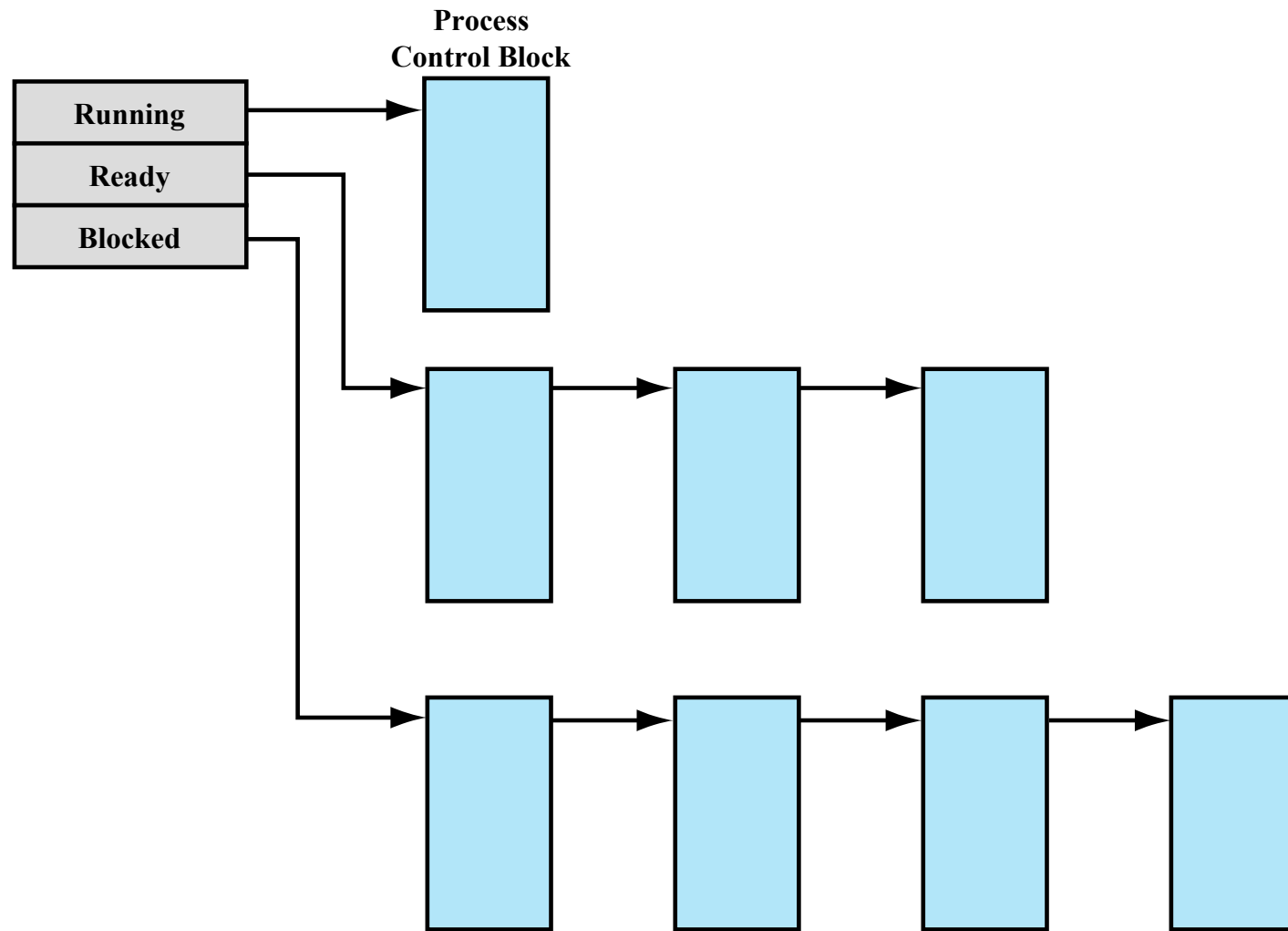


Figure 3.14 Process List Structures

Role of the PCB

- Important data structure in an OS
 - Contains all of the information about a process that is needed by the OS
 - PCBs are read and/or modified by virtually every module in the OS
 - Defines the state of the OS
 - The OS's behavior (scheduling, switching processes, responding to events) depends entirely on the information in the PCBs.
- Difficulty is not access, but protection
 - A bug in a single routine could damage process control blocks, which could destroy the system's ability to manage the affected processes
 - A design change in the structure or semantics of the process control block could affect a number of modules in the OS
- A Handler can be used which takes care of writing and reading PCBs
 - Trade-off between performance and trust.

Process Control

- Let's look at modes of execution. (quick recap)
- We looked at this concept in the overview.

Modes of execution

- User Mode
 - Less-privileged mode
 - User programs typically execute in this mode
- System Mode
 - More-privileged mode
 - Also referred to as control mode or kernel mode
 - Kernel of the operating system
 - Kernel mode/control mode
 - (Kernel is the core of the operating system)

Modes of execution

- Why?
 - To protect the OS and key operating system tables, such as PCB, page tables, from interference by user programs and malicious actors.
- How does the processor know which mode to be in?
 - Typically, PSW indicates the mode of execution.
 - A bit in PSW changes in response to certain events.
 - user program makes a **call to an OS service**.
 - Interrupt triggers
 - Upon returning from such services, to user process this bit is set to user mode.
 - Interrupt handling routine contains IRT (interrupt return) at the end.
 - When issuing a system call user program places the system call identifier and system call arguments in a predefined area, then executes a special instruction that has the effect of interrupting execution at the user level and transferring control to the kernel.

Modes of execution

- Most processors supports at least two modes of executions.
 - x86 (Intel/AMD) Supports 4 privilege levels (rings 0–3):
 - Ring 0 → kernel mode (full access)
 - Ring 1–2 → rarely used, sometimes for drivers or hypervisors
 - Ring 3 → user mode

Typical functions of an OS kernel

Process management

- Process creation and termination
- Process scheduling and dispatching
- Process switching
- Process synchronization and interprocess communication
- Management of process control blocks

Memory management

- Allocation of address space to processes
- Swapping
- Page and segment management

I/O management

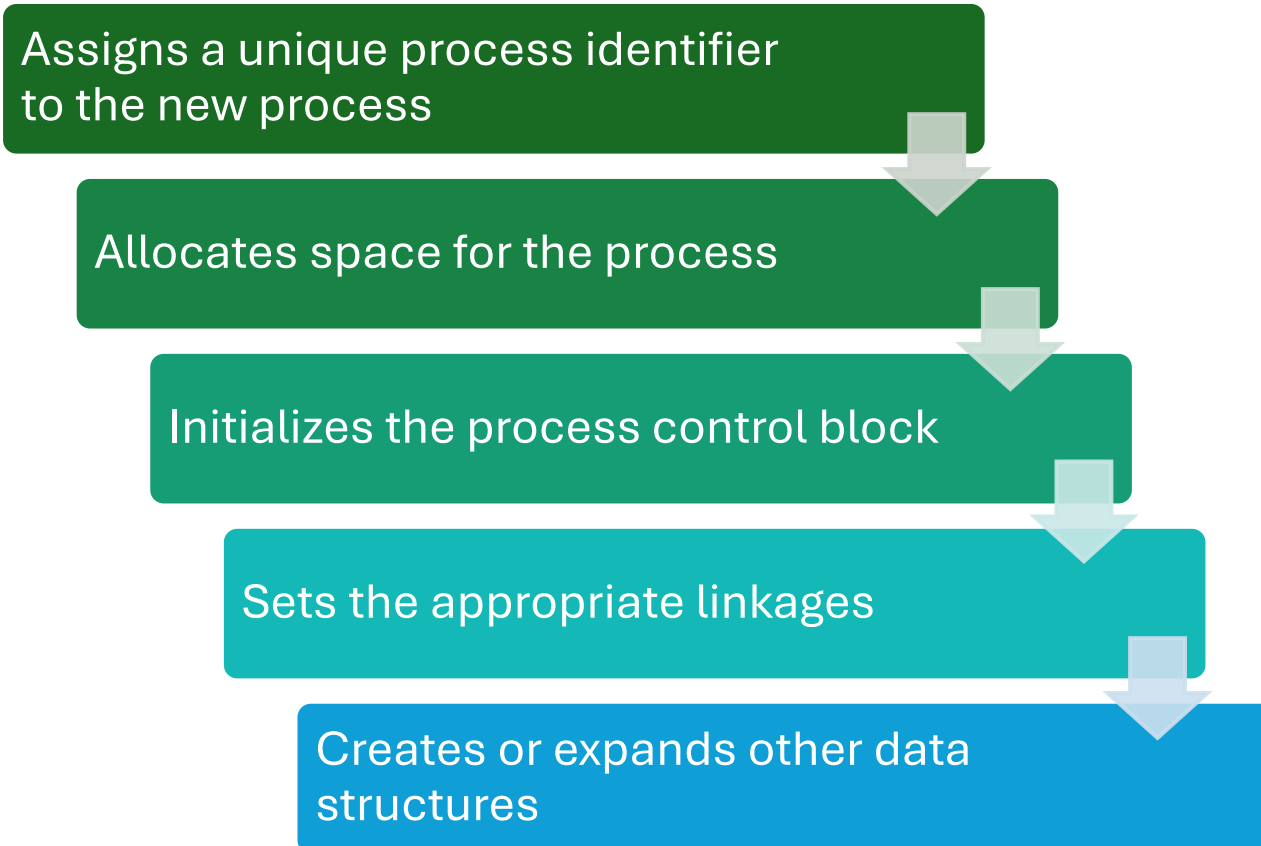
- Buffer management
- Allocation of I/O channels and devices to processes

Support functions

- Interrupt handling
- Accounting
- Monitoring

Process creation

- Once the OS decides to create a new process it:



Process switching

- When a running process is interrupted, OS assigns another process to the running state and turns control to that process.
- Problems consider:
 1. Which events trigger a process switch?
 2. Mode switching and process switching
 3. What happens to the control structures?

Mechanisms for interrupting a process

Mechanism	Cause	Use
Interrupt	External to the execution of the current instruction	Reaction to an asynchronous external event
Trap	Associated with the execution of the current instruction	Handling of an error or an exception condition
Supervisor call	Explicit request	Call to an operating system function

System interrupts (2 types)

- Interrupt

- Due to some sort of event that is external to and independent of the currently running process
 - Clock interrupt
 - I/O interrupt
 - Memory fault
- Time slice
 - The maximum amount of time that a process can execute before being interrupted

- Trap

- An error or exception condition generated within the currently running process
- OS determines if the condition is fatal
- Moved to the Exit state and a process switch occurs
- Action will depend on the nature of the error the design of the OS

Mode switching

If no interrupts are pending the processor:



Proceeds to the fetch stage and fetches the next instruction of the current program in the current process

If an interrupt is pending the processor:



Sets the program counter to the starting address of an interrupt handler program



Switches from user mode to kernel mode so that the interrupt processing code may include privileged instructions

Change of process state

